

Alterando a interface com useState

Projeto: Painel de Configurações - estados booleanos, texto, tema e visibilidade.

Objetivo

Entender que useState pode controlar mais do que números.

Vamos usar

useState, componentes, props, children, onClick e condicionais simples.

Resultado

A tela terá controles para notificações, tema claro/escuro e perfil visível/escondido.

Passo 1 - Estado booleano

Boolean guarda somente true ou false.

Exemplo

```
const [ligado, setLigado] =  
  useState(false);
```

false

Desligado, escondido ou desativado.

true

Ligado, visível ou ativado.

Passo 2 - Inverter o valor

O operador ! troca true por false e false por true.

Exemplo

```
function alterar() {  
  setLigado(!ligado);  
}
```

Leitura

Se ligado estiver false, !ligado vira true. Se estiver true, !ligado vira false.

Passo 3 - Criar CardConfiguracao.jsx

Revisão de props e children.

```
src/components/CardConfiguracao.jsx

function CardConfiguracao({ titulo, children }) {
  return (
    <section className="card">
      <h2>{titulo}</h2>

      <div className="conteudo-card">
        {children}
      </div>
    </section>
  );
}

export default CardConfiguracao;
```

children

O conteúdo colocado entre e será exibido dentro do card.

Passo 4 - Criar os três estados

Cada estado controla uma parte diferente da tela.

 Dentro de App()

```
const [notificacoes, setNotificacoes] =  
  useState(true);  
  
const [temaEscuro, setTemaEscuro] =  
  useState(false);  
  
const [perfilVisivel, setPerfilVisivel] =  
  useState(true);
```

Passo 5 - Criar as funções

Cada botão altera apenas o estado correspondente.

```
Dentro de App()  
  
function alterarNotificacoes() {  
  setNotificacoes(!notificacoes);  
}  
  
function alterarTema() {  
  setTemaEscuro(!temaEscuro);  
}  
  
function alterarPerfil() {  
  setPerfilVisivel(!perfilVisivel);  
}
```

Passo 6 - Alterar uma classe com o estado

O tema da página será escolhido pelo valor de temaEscuro.

Dentro do return

```
<main
  className={
    temaEscuro
      ? "app escuro"
      : "app claro"
  }
>
  { /* conteúdo */ }
</main>
```

Ternário

condição ? valor quando true : valor quando false

Passo 7 - Alterar textos com o estado

Notificações muda o status e também o texto do botão.

Dentro de <CardConfiguracao>

```
<p>
  Status:
  {notificacoes
    ? " Ativadas"
    : " Desativadas"}
</p>

<button onClick={alterarNotificacoes}>
  {notificacoes
    ? "Desativar"
    : "Ativar"}
</button>
```

Passo 8 - Mostrar ou esconder um elemento

perfilVisivel decide se o bloco de perfil aparece.

Dentro de <CardConfiguracao>

```
{perfilVisivel && (  
  <div className="perfil">  
    <h3>Perfil do usuário</h3>  
    <p>Estudante React</p>  
  </div>  
)}
```

&& no JSX

Se perfilVisivel for true, o bloco aparece. Se for false, ele não é renderizado.

Passo 9 - O fluxo da interface dinâmica

O mesmo padrão será usado em todo o projeto.

Fluxo

clique -> função -> setEstado() -> estado muda -> React renderiza novamente

Texto

Pode mudar conforme o estado usando o operador ternário.

Elemento

Pode aparecer ou desaparecer usando &&.

Estilo

Uma classe CSS também pode ser escolhida a partir do estado, como claro ou escuro.

Passo 10 - App.jsx completo (1/4)

Copie as partes na ordem. É um único arquivo.

src/App.jsx

```
import { useState } from "react";
import CardConfiguracao from "../components/CardConfiguracao";
import "../App.css";

function App() {
  const [notificacoes, setNotificacoes] = useState(true);
  const [temaEscuro, setTemaEscuro] = useState(false);
  const [perfilVisivel, setPerfilVisivel] = useState(true);

  function alterarNotificacoes() {
    setNotificacoes(!notificacoes);
  }

  function alterarTema() {
    setTemaEscuro(!temaEscuro);
  }

  function alterarPerfil() {
```

Passo 11 - App.jsx completo (2/4)

Copie as partes na ordem. É um único arquivo.

src/App.jsx

```
    setPerfilVisivel(!perfilVisivel);
  }

  return (
    <main className={temaEscuro ? "app escuro" : "app claro"}>
      <h1>Painel de Configurações</h1>

      <CardConfiguracao titulo="Notificações">
        <p>
          Status: {notificacoes ? "Ativadas" : "Desativadas"}
        </p>

        <button onClick={alterarNotificacoes}>
          {notificacoes ? "Desativar" : "Ativar"}
        </button>
      </CardConfiguracao>

      <CardConfiguracao titulo="Tema">
```

Passo 12 - App.jsx completo (3/4)

Copie as partes na ordem. É um único arquivo.

src/App.jsx

```
<p>
  Tema atual: {temaEscuro ? "Escuro" : "Claro"}
</p>

<button onClick={alterarTema}>
  Alterar tema
</button>
</CardConfiguracao>

<CardConfiguracao titulo="Perfil">
  <p>
    Status: {perfilVisivel ? "Visível" : "Escondido"}
  </p>

  <button onClick={alterarPerfil}>
    {perfilVisivel
      ? "Esconder perfil"
      : "Mostrar perfil"}
  </button>
</CardConfiguracao>
```

Passo 13 - App.jsx completo (4/4)

Copie as partes na ordem. É um único arquivo.

src/App.jsx

```
    </button>

    {perfilVisivel && (
      <div className="perfil">
        <h3>Perfil do usuário</h3>
        <p>Estudante React</p>
      </div>
    )}
  </CardConfiguracao>
</main>
);
}

export default App;
```

Passo 14 - App.css (1/3)

CSS simples para enxergar as mudanças de tema e perfil.

```
src/App.css

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, Helvetica, sans-serif;
}

.app {
  min-height: 100vh;
  padding: 40px;
  transition: 0.3s;
}

.app h1 {
  text-align: center;
}

.claro {
```

Passo 15 - App.css (2/3)

CSS simples para enxergar as mudanças de tema e perfil.

src/App.css

```
background: #f4f4f4;
color: #111111;
}

.escuro {
  background: #080808;
  color: #ffffff;
}

.card {
  max-width: 650px;
  margin: 20px auto;
  padding: 22px;
  border: 1px solid #777777;
  border-radius: 12px;
}

.card h2 {
  color: #ff7a00;
}
```

Passo 16 - App.css (3/3)

CSS simples para enxergar as mudanças de tema e perfil.

src/App.css

```
button {
  padding: 10px 16px;
  background: #ff7a00;
  color: #111111;
  border: 0;
  border-radius: 8px;
  font-weight: bold;
  cursor: pointer;
}

.perfil {
  margin-top: 18px;
  padding: 16px;
  border: 1px solid #ff7a00;
  border-radius: 10px;
}
```

Passo 17 - Testar o projeto

Confira cada alteração separadamente.

Notificações

Ativar/Desativar deve mudar o texto e o botão.

Tema

Alterar tema deve trocar entre claro e escuro.

Perfil

Mostrar/Esconder deve controlar o bloco de perfil.

Conclusão

O estado controla diretamente o que o usuário vê.

Ideia principal da Aula 17

useState guarda um valor. Quando setEstado() altera esse valor, o React renderiza novamente e a interface pode mudar.